

Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas



ELI NOSQL

PONDERACIÓN: 45

Horas Aproximadas: 80

Índice

1. Resumen Ejecutivo	3
2. Competencia que desarrollaremos	3
3. Objetivos del Aprendizaje	3
3.1. Objetivo General	4
3.2. Objetivos Específicos	4
4. Enunciado del Proyecto	4
4.1 Descripción del problema a resolver	4
5. Descripción del Lenguaje	6
5.1 Comentarios	7
5.2 Tipos de Datos	7
5.3 Operadores Relacionales	8
5.4 Uso de Operadores Relacionales	9
5.5 Operadores Lógicos	9
5.6 Signos de Agrupación	10
5.7 Precedencia de Operadores	11
5.8 Instrucciones de Persistencia y Exportación	12
5.8 Definición de Base de Datos con Persistencia	13
5.9 Palabras Reservadas e Instrucciones del Lenguaje ELI	14
5.10 Instrucciones de Gestión del Lenguaje ELI	17
5.11 Estructura JSON para Persistencia de Base de Datos	20
5.12 Estructura JSON para Exportación de Consultas	20
5.13 Carga de Información Persistida	21
6. Reportes	21
6.1 Tabla de errores	21
6.2 Tabla de Tokens	22
7. Alcance del proyecto	22
8. Requerimientos técnicos	22
9. Entregables	23
10. Metodología	23
11. Desarrollo de Habilidades Blandas	23
12. Cronograma	24
13. Rúbrica de Calificación	24
13.1 Detalle de la Calificación	24
13.2 Valores	24
13.3 Comentarios Generales	25

1. Resumen Ejecutivo

El curso de Organización de Lenguajes y Compiladores 1, perteneciente a la Facultad de Ingeniería de la Universidad de San Carlos de Guatemala, necesita un sistema capaz de gestionar datos y reconocer un nuevo lenguaje de programación llamado ELI y generar reportes.

El lenguaje ELI presenta una **sintaxis original**, inspirada en el dominio de bases de datos, pero **no compatible con SQL**, lo que permite a los estudiantes definir estructuras de datos, realizar inserciones, consultas, modificaciones y eliminaciones mediante construcciones propias del lenguaje. El procesamiento del lenguaje se realiza de manera completamente independiente del motor de almacenamiento.

El propósito principal del proyecto es que el estudiante **diseñe, implemente y documente** las fases iniciales de un compilador, aplicando de manera práctica los conceptos vistos en el curso **Organización de Lenguajes y Compiladores 1**.

2. Competencia que desarrollaremos

A través del desarrollo del proyecto **ELI (Educational Language Implementation)**, el estudiante desarrollará las siguientes competencias académicas y técnicas:

1. **Implementación de analizadores léxicos**, aplicando expresiones regulares y herramientas especializadas como **JFlex** para la tokenización de lenguajes formales y el manejo de errores léxicos.
2. **Diseño y gestión de entradas**, incorporando manejo de ámbitos, registro de entidades del lenguaje y organización estructurada de la información semántica básica.
3. Construcción de analizadores sintácticos, utilizando **CUP** para el diseño de gramáticas LL y LR, validando la estructura formal de programas y comprendiendo los procesos de derivación por la izquierda y por la derecha.

3. Objetivos del Aprendizaje

3.1. Objetivo General

Desarrollar una **aplicación gráfica educativa** que implemente el **front-end de un compilador** para el lenguaje **ELI (Educational Language Implementation)**, permitiendo a los estudiantes **diseñar, analizar y validar** un lenguaje declarativo propio mediante la aplicación práctica del **análisis léxico**, la **recuperación de errores**, la gestión de **tablas de**

símbolos, con el fin de fortalecer la comprensión de los conceptos fundamentales del curso **Organización de Lenguajes y Compiladores 1**.

3.2. Objetivos Específicos

Al finalizar el proyecto, los estudiantes deberán ser capaces de:

1. Reforzar los conocimientos de análisis léxico para la creación de un lenguaje de programación como la primera fase de análisis del compilador.
2. Aplicar los conceptos de compiladores para implementar un proceso de interpretación de código de alto nivel.
3. Aplicar los conceptos de compiladores para analizar un lenguaje de programación y producir las salidas esperadas.
4. Aplicar la teoría de compiladores para la creación de soluciones de software.

4. Enunciado del Proyecto

4.1 Descripción del problema a resolver

El curso de **Organización de Lenguajes y Compiladores 1**, perteneciente a la **Facultad de Ingeniería de la Universidad de San Carlos de Guatemala**, requiere el desarrollo de una **aplicación educativa** capaz de **gestionar datos e interpretar programas escritos en el lenguaje ELI**. Esta aplicación permitirá a los usuarios **editar y analizar código, generar y visualizar reportes, y mostrar los datos almacenados** como resultado de la ejecución del lenguaje.

El sistema contará con un **entorno gráfico de desarrollo**, que incluirá un editor de código, un módulo de análisis léxico, y un área de salida donde se presentarán los resultados de las operaciones realizadas sobre los datos. Asimismo, la aplicación será capaz de **persistir la información almacenada en memoria**, permitiendo conservar el estado de los datos durante la ejecución de la aplicación, sin delegar el análisis del lenguaje a sistemas externos.

Se recomienda que se desarrolle mediante una arquitectura capaz.



Entorno de Trabajo

Editor

El editor será parte del entorno de trabajo, cuya finalidad será proporcionar ciertas funcionalidades, características y herramientas que serán de utilidad al usuario. La función principal del editor será el ingreso de código fuente que será analizado. En este se podrán abrir diferentes archivos al mismo tiempo.

Funcionalidades

- **Crear archivos:** El editor deberá ser capaz de crear archivos en blanco
- **Abrir archivos:** El editor deberá ser capaz de abrir archivos
- **Guardar archivo:** El editor deberá guardar el estado del archivo en el que se está trabajando.

Herramientas

- **Ejecutar:** La opción Ejecutar realizará el llamado al intérprete del lenguaje ELI, el cual será el encargado de llevar a cabo el análisis léxico del código fuente.

Reportes

- **Reporte de Errores:** se mostrarán todos los errores encontrados al realizar el análisis léxico. Cada error deberá incluir la siguiente información: Tipo de Error, Descripción, Línea, Columna

- Reporte tabla de tokens: se debe de generar un reporte con todos los tokens identificados.

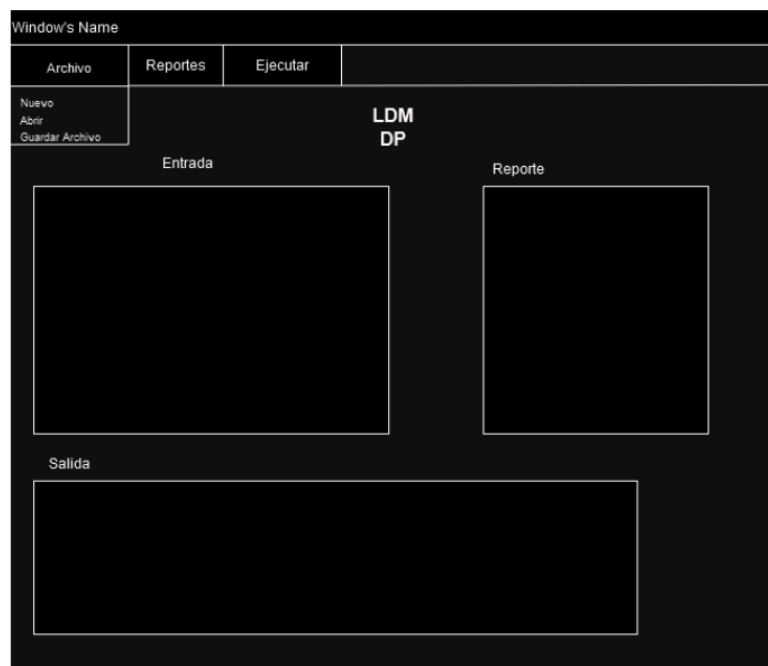
IDE:

Área de Consola:

En esta área se mostrarán los resultados, mensajes y todo lo que sea indicado dentro del lenguaje

Output:

En esta área se deberán mostrar el resultado de las consultas realizadas.



5. Descripción del Lenguaje

5.1 Comentarios

ELI soporta dos tipos de comentarios: comentarios de una línea y comentarios multilínea.

Comentarios de una línea:

Los comentarios de una línea se utilizan para documentar una instrucción o agregar anotaciones breves dentro del código. Este tipo de comentario inicia con una secuencia `##` y se extiende hasta el final de la línea en la que se encuentra. Todo el texto que aparezca después de `##` será considerado un comentario y no será procesado por el intérprete.

```
## Este es un comentario de linea.
database universidad{
}
```

Comentarios Multilínea:

Los comentarios multilínea se utilizan para documentar bloques completos de código o agregar explicaciones más extensas. Este tipo de comentario inicia con la secuencia `/*` y finaliza con la secuencia `*/`. Todo el contenido que se encuentre entre estos delimitadores será ignorado por el intérprete.

```
/* Este es un comentario multilínea.
   tabla que registra entidades tipo estudiante
*/
table estudiantes{
  id: int;
  nombre: string
  edad: int; }
```

Nota: Los comentarios multilínea pueden abarcar una o varias líneas de código.

5.2 Tipos de Datos

Los tipos de dato que soportará el lenguaje en concepto de un tipo de variable se definen a continuación:

Tipo	Descripción	Ejemplo	Observaciones	Valor por defecto
Entero (int)	Admite únicamente valores numéricos enteros.	1, -120	Se maneja cualquier cantidad de dígitos.	0
Decimal (float)	Admite valores numéricos con parte decimal.	1.5, -3.2	Permite decimales positivos y negativos.	0.0
Booleano (bool)	Admite valores lógicos verdadero o falso.	true, false	Utilizado en filtros y evaluaciones lógicas.	false
Cadena (string)	Conjunto de caracteres delimitado por comillas dobles.	"Luis", "ELI"	Permite secuencias de escape.	""
Arreglo (array)	Colección ordenada de valores del mismo o distinto tipo.	[1, 2, 3]	Puede contener tipos mixtos.	[]

Objeto (object)	Conjunto de pares clave-valor.	{ nombre: "Luis", edad: 22 }	Similar a documentos NoSQL.	{ }
Nulo (null)	Representa la ausencia de un valor.	null	Útil para campos opcionales.	null

5.3 Operadores Relacionales

El lenguaje **ELI** soporta únicamente **operadores relacionales**, los cuales se utilizan para **comparar valores** dentro de expresiones de filtrado, principalmente en la cláusula **filter** de las sentencias de lectura.

ELI **no soporta operadores aritméticos**, ya que el lenguaje está orientado a la **consulta y manipulación de datos**, y no al cálculo numérico.

Las expresiones relacionales permiten evaluar condiciones lógicas que determinan qué registros cumplen con un criterio específico.

Operador	Descripción	Ejemplo
==	Evalúa si dos valores son iguales	edad == 20
!=	Evalúa si dos valores son diferentes	edad != 18
>	Evalúa si un valor es mayor que otro	edad > 21
<	Evalúa si un valor es menor que otro	edad < 30
>=	Evalúa si un valor es mayor o igual que otro	edad >= 18
<=	Evalúa si un valor es menor o igual que otro	edad <= 65

5.4 Uso de Operadores Relacionales

Los operadores relacionales se utilizan exclusivamente dentro de expresiones de filtrado para comparar **campos de una tabla** contra **valores literales**.

Ejemplo:

```
read estudiantes {
  fields: nombre, edad,
  filter: edad > 20
};
```


5.5 Operadores Lógicos

El lenguaje **ELI** soporta **operadores lógicos** con el objetivo de permitir la **combinación de expresiones relacionales** dentro de las cláusulas de filtrado. Estos operadores permiten construir condiciones más complejas para la selección de datos, manteniendo la simplicidad y claridad del lenguaje.

Los operadores lógicos se utilizan **exclusivamente dentro de la cláusula **filter**** y operan únicamente sobre expresiones relacionales. El resultado de una expresión lógica es siempre un **valor booleano**.

Operador	Descripción	Ejemplo
&&	Operador lógico AND. Evalúa verdadero si ambas expresiones son verdaderas.	<code>edad > 20 && edad < 30</code>
!	Operador lógico NOT. Niega el valor de una expresión.	<code>!(edad < 18)</code>
	Operador lógico OR. Evalúa verdadero si una de ambas expresiones es verdadera.	<code>edad > 20 nombre != "Ana"</code>

Ejemplo:

```
read estudiantes {
  fields: nombre, edad,
  filter: edad > 20 && edad < 30
};
```

```
read estudiantes {
  fields: nombre,
  filter: nombre == "Luis" || nombre == "Ana"
};
```

```
read estudiantes {
  fields: nombre, edad,
  filter: !(edad < 18)
};
```

Restricciones

- Los operadores lógicos solo pueden utilizarse dentro de la cláusula **filter**.
- No se permite el uso de operadores aritméticos.
- Los operadores lógicos deben combinar únicamente expresiones relacionales.
- No se permite el uso de expresiones lógicas fuera del contexto de filtrado.

5.6 Signos de Agrupación

El lenguaje **ELI** utiliza **signos de agrupación** para **delimitar bloques de instrucciones**, **definir estructuras** y **controlar la precedencia de evaluación** dentro de las expresiones lógicas. Estos símbolos permiten organizar el código de forma clara y estructurada, facilitando su análisis léxico.

Símbolo	Descripción	Uso principal
{ }	Llaves	Delimitan bloques de definición y estructuras del lenguaje
()	Paréntesis	Agrupar expresiones lógicas dentro de filtros
[]	Corchetes	Delimitan arreglos de valores

Uso de Llaves { }

Las llaves se utilizan para definir **bloques estructurales** dentro del lenguaje ELI, tales como la definición de bases de datos, tablas y sentencias que contienen conjuntos de atributos o parámetros.

Ejemplo:

```
table estudiantes {  
  id : int;  
  nombre : string;  
  edad : int;  
}
```

Uso de Paréntesis ()

Los paréntesis se utilizan para **agrupar expresiones lógicas** y controlar la **precedencia de evaluación** dentro de la cláusula **filter**.

Ejemplo:

```
read estudiantes {  
  fields: nombre, edad,
```

```
filter: (edad > 20 && edad < 30) || nombre == "Luis"
};
```

Uso de Corchetes []

Los corchetes se utilizan para definir arreglos de valores, permitiendo almacenar múltiples elementos dentro de un mismo campo.

```
add estudiantes {
  id: 1,
  nombre: "Luis",
  cursos: ["Compiladores", "Bases de Datos", "IA"]
};
```

Restricciones

- Los signos de agrupación deben abrirse y cerrarse correctamente.
- No se permite el uso de signos de agrupación fuera de los contextos definidos por el lenguaje.

5.7 Precedencia de Operadores

El lenguaje **ELI** define un conjunto de reglas de **precedencia de operadores** que determinan el orden en el que se evalúan las expresiones dentro de la cláusula **filter**. Estas reglas permiten interpretar correctamente expresiones que combinan **operadores relacionales**, **operadores lógicos** y **signos de agrupación**.

Cuando una expresión contiene múltiples operadores, aquellos con **mayor precedencia** se evalúan antes que los de menor precedencia. En caso de ambigüedad, el uso de **paréntesis** permite alterar explícitamente el orden de evaluación.

Nivel	Operador	Descripción
1	()	Agrupación explícita de expresiones
2	!	Operador lógico NOT
3	>, <, >=, <=, ==, !=	Operadores relacionales
4	&&	Operador lógico AND
5		Operador lógico OR

Asociatividad

- El operador lógico **NOT (!)** es de **asociatividad derecha**.
- Los operadores relacionales son de **asociatividad izquierda**.
- Los operadores lógicos **AND (&&)** y **OR (||)** son de **asociatividad izquierda**.

Ejemplos de Evaluación

```
filter: edad > 20 && edad < 30 || nombre == "Luis"
```

La evaluación se realiza de la siguiente forma:

1. Se evalúan las expresiones relacionales (`edad > 20`, `edad < 30`, `nombre == "Luis"`).
2. Se evalúa el operador `&&`.
3. Finalmente, se evalúa el operador `||`.

5.8 Instrucciones de Persistencia y Exportación

El lenguaje ELI incorpora un conjunto de instrucciones orientadas a la persistencia de información y exportación de resultados, permitiendo almacenar y recuperar datos mediante archivos en formato JSON, sin utilizar motores externos de bases de datos.

Estas instrucciones permiten conservar la información generada durante la ejecución del programa y reutilizarla en ejecuciones posteriores.

Instrucción `export`

Descripción

La instrucción `export` permite guardar en un archivo JSON los registros obtenidos como resultado de una consulta (`read`).

El archivo generado deberá contener tanto los datos consultados como la estructura de la consulta, incluyendo los tipos de datos asociados a cada campo.

Estructura

```
export <nombre_archivo>.json;
```

Ejemplo

```
read estudiantes{
    fields: nombre, edad,
    filter: edad > 20
};

export estudiantes_mayores.json;
```

Comportamiento

- La instrucción `export` debe ejecutarse después de una consulta válida.
- El archivo JSON generado contendrá:
 - Nombre de la tabla consultada
 - Campos seleccionados
 - Tipos de datos
 - Registros resultantes
- Si no existe una consulta previa, se reportará un error.

5.8 Definición de Base de Datos con Persistencia

Descripción

El lenguaje ELI permite definir una base de datos asociada a un archivo de persistencia, en el cual se almacenará toda la información estructural y los registros definidos dentro de dicha base de datos.

El alcance de las declaraciones queda limitado a la base de datos definida, y no se permite la interacción entre múltiples bases de datos dentro del mismo programa.

Estructura

```
database <nombre_base> {
    store at "<ruta_archivo>.json";
    // Declaraciones de tablas
}
```

Ejemplo

```
database universidad {
    store at "universidad.json";
}
table estudiantes {
    id : int;
    nombre : string;
    edad : int;
}
```

5.9 Palabras Reservadas e Instrucciones del Lenguaje ELI

El lenguaje **ELI** define un conjunto de **palabras reservadas** que permiten la creación, manipulación y consulta de datos mediante una sintaxis declarativa propia. Estas palabras reservadas no pueden ser utilizadas como identificadores y poseen un significado específico dentro del lenguaje.

database

Descripción

La instrucción `database` se utiliza para **definir una base de datos lógica** dentro del programa. Esta definición establece el **contexto principal de trabajo**, sobre el cual se podrán crear tablas, insertar datos y realizar consultas.

El bloque asociado a la instrucción `database` **no contiene declaraciones de tablas**. Dentro de este bloque únicamente se permite la instrucción `store at`, la cual define la ruta de persistencia de la base de datos.

Si no se especifica la instrucción `store at`, la base de datos será almacenada en una **ruta por defecto definida por el estudiante**.

Estructura

```
database <nombreBaseDatos> {  
    store at "ruta/archivo.json";  
}
```

Ejemplo

```
database universidad {  
    store at "universidad.json";  
}
```

use

Descripción

La instrucción `use`, se utiliza para **seleccionar una base de datos activa** sobre la cual se ejecutarán las operaciones posteriores del programa.

Una vez seleccionada una base de datos mediante `use`, todas las instrucciones de **creación de tablas, inserción de registros, consultas, exportaciones y gestión de datos** se realizarán dentro del contexto de dicha base de datos.

Solo puede existir **una base de datos activa a la vez**.

Estructura

```
use <nombreBaseDatos>;
```

Ejemplo

```
use universidad;
```

table

Descripción

La palabra reservada **table** se utiliza para **definir una estructura de almacenamiento** dentro de la base de datos. Una tabla representa una colección de registros que comparten el mismo conjunto de campos.

Cada tabla debe declararse para una base de datos utilizando la palabra reservada **use** (se especifica mas adelante) y sus campos deben especificar obligatoriamente un tipo de dato.

Estructura

```
table <nombreTabla> {  
    <campo> : <tipo>;  
    ...  
}
```

Ejemplo

```
table estudiantes {  
    id : int;  
    nombre : string;  
    edad : int;  
}
```

read

Descripción

La instrucción **read** se utiliza para **consultar los registros almacenados en una tabla**. Permite seleccionar campos específicos y aplicar condiciones de filtrado sobre los datos.

El resultado de una instrucción **read** se muestra en el área de salida del sistema y puede ser utilizado posteriormente por la instrucción **export**.

Estructura

```
read <nombreTabla> {  
    fields: <listaCampos>;  
    filter: <expresionRelacional>;  
};
```

Ejemplo

```
read estudiantes {  
    fields: nombre, edad;  
    filter: edad > 20;  
};
```

fields

Descripción

La palabra reservada `fields` se utiliza dentro de una instrucción `read` para **indicar qué campos de la tabla deben mostrarse** como resultado de la consulta.

Estructura

```
fields: campo1, campo2, ...;
```

También se permite seleccionar todos los campos mediante el comodín `*`.

Ejemplo

```
fields: *;
```

filter

Descripción

La palabra reservada `filter` se utiliza para **definir una condición de filtrado** sobre los registros de una tabla. Esta condición determina qué registros cumplen con el criterio especificado y deben ser incluidos en el resultado de la consulta.

Las expresiones de filtrado pueden incluir **operadores relacionales y lógicos**, respetando la precedencia definida por el lenguaje.

Estructura

```
filter: <expresion>;
```

Ejemplo

```
filter: edad > 20 && edad < 30;
```

store at

Descripción

La instrucción `store at` se utiliza para **definir el archivo de persistencia** asociado a una base de datos. En dicho archivo se almacenará tanto la estructura como los registros de la base de datos.

Esta instrucción debe declararse **dentro del bloque `database`** y solo puede aparecer una vez.

Estructura

```
store at "archivo.json";
```

export

Descripción

La instrucción `export` se utiliza para **guardar en un archivo JSON los registros obtenidos como resultado de la última consulta (`read`) ejecutada**. El archivo generado contendrá tanto los datos consultados como su estructura y tipos de datos.

Estructura

```
export "archivo.json";
```

5.10 Instrucciones de Gestión del Lenguaje ELI

El lenguaje **ELI** incorpora un conjunto de instrucciones adicionales orientadas a la **gestión y administración de las estructuras de datos**, permitiendo manipular tablas, visualizar su estructura y controlar la persistencia de la información almacenada.

Estas instrucciones complementan las operaciones básicas del lenguaje y están diseñadas para mantener la simplicidad y claridad del modelo de datos.

add

Descripción

La instrucción `add` se utiliza para **insertar un nuevo registro** dentro de una tabla existente. Los valores se especifican mediante pares clave–valor, donde cada clave corresponde a un campo de la tabla.

Estructura

```
add <nombreTabla> {  
    campo: valor,  
    ...  
};
```

Ejemplo

```
add estudiantes {  
    id: 1,  
    nombre: "Luis",  
    edad: 22  
};
```

update

Descripción

La instrucción **update** se utiliza para **modificar los valores de uno o más campos** de los registros almacenados en una tabla.

La actualización se realiza únicamente sobre los registros que **cumplan una condición de filtrado**, permitiendo cambios controlados dentro de la base de datos activa.

La instrucción **update** **no altera la estructura de la tabla**, únicamente modifica los valores de los campos especificados.

Estructura

```
update <nombreTabla> {  
    set: campo1 = valor1, campo2 = valor2;  
    filter: <expresion>;  
};
```

Ejemplo

```
update estudiantes {  
    set: edad = 23, nombre = "Luis Pérez"  
    filter: id == 1;  
};
```

Descripción de los Componentes

- **set**
Indica los campos que serán actualizados y los nuevos valores que se les asignarán. Cada asignación se define mediante la sintaxis **campo = valor**.
- **filter**
Define la condición que deben cumplir los registros para ser actualizados. La expresión puede incluir operadores relacionales y lógicos, respetando la precedencia definida por el lenguaje.

Ejemplo sin Filtro

Si se omite la cláusula `filter`, **todos los registros de la tabla serán actualizados**.

```
update estudiantes {  
    set: activo = false  
};
```

clear

Descripción

La instrucción `clear` se utiliza para **eliminar todos los registros almacenados en una tabla**, sin afectar su estructura ni su definición.

Después de ejecutar esta instrucción, la tabla conserva sus campos y tipos de datos, pero no contiene registros.

Estructura

```
clear <nombreTabla>;
```

Ejemplo

```
clear estudiantes;
```

Restricciones

- La ruta del archivo debe ser una cadena válida.
- Solo puede definirse una base de datos activa.
- Todas las tablas y registros pertenecen exclusivamente a dicha base de datos.

5.11 Estructura JSON para Persistencia de Base de Datos

El archivo JSON asociado a una base de datos ELI almacenará toda la información necesaria para reconstruirla, incluyendo estructura y datos.

Estructura General

```
{  
  "database": "universidad",  
  "tables": {  
    "estudiantes": {  
      "schema": {  
        "id": "int",  
        "nombre": "string",  
        "edad": "int"  
      }  
    }  
  }  
}
```

```

    },
    "records": [
      {
        "id": 1,
        "nombre": "Luis",
        "edad": 22
      },
      {
        "id": 2,
        "nombre": "Ana",
        "edad": 19
      }
    ]
  }
}

```

5.12 Estructura JSON para Exportación de Consultas

Cuando se ejecuta una instrucción **export**, se genera un archivo JSON con la siguiente estructura:

```

{
  "table": "estudiantes",
  "fields": {
    "nombre": "string",
    "edad": "int"
  },
  "records": [
    {
      "nombre": "Luis",
      "edad": 22
    }
  ]
}

```

Contenido del Archivo de Exportación

- **table**: tabla consultada.
- **fields**: campos seleccionados con sus tipos de datos.
- **records**: registros que cumplen con el filtro aplicado.

5.13 Carga de Información Persistida

Al ejecutar la aplicación y detectar un archivo de base de datos previamente creado mediante `store at`, el sistema deberá:

1. Leer el archivo JSON.
2. Reconstruir las estructuras en memoria.
3. Permitir consultas (`read`) sobre los registros almacenados.

6. Reportes

Los reportes son parte fundamental de ELI-NOSQL, ya que muestra de forma visual las herramientas utilizadas para realizar la ejecución del código.

A continuación, se muestran ejemplos de estos reportes. Queda a discreción del estudiante el diseño de estos.

6.1 Tabla de errores

El reporte de errores debe contener la información suficiente para detectar y corregir errores en el código fuente.

#	Tipo	Descripción	Línea	Columna
1	Léxico	El carácter "\$" no pertenece al lenguaje	5	3
2	Léxico	La palabra "Def" no pertenece al lenguaje.	6	3
3	Léxico	El carácter "#" no pertenece al lenguaje.	12	5

6.2 Tabla de Tokens

El reporte de tokens debe tener la información suficiente para poder identificar los tokens reconocidos en el análisis léxico. Este debe ser en una tabla.

#	Lexema	Tipo	Línea	Columna
1	arr	id	5	3
2	3	Double	6	3
3	"Hola"	String	8	3

7. Alcance del proyecto

El software debe poder mostrar los resultados de las operaciones relacionales y logicas. Así como la oportunidad de generar Reportes de Errores lexicos y la Tabla de Símbolos utilizada.

8. Requerimientos técnicos

- Lenguaje de programación
- Herramienta para generar lexer y parser
- Librería de visualización
- Interfaz gráfica
- Entrada de archivos .code.

9. Entregables

Tipo	Descripción
Código Fuente	Link de su repositorio en Github para la entrega del código fuente usado en el proyecto.
Manual Técnico	Información importante del proyecto para que se pueda realizar el mantenimiento en el futuro. Especificar el lenguaje, herramientas utilizadas, métodos y funciones más importantes.
Manual de Usuario	Documento que explica cómo usar el sistema desarrollado, incluyendo capturas de pantalla y pasos detallados

Archivo de Gramática en un archivo Markdown	El archivo debe contener su gramática y debe de ser limpio, entendible y no debe ser una copia del archivo de Jison. La gramática debe estar escrita en formato BNF(Backus-Naur form).Solamente se debe escribir la gramática independiente del contexto.
--	---

10. Metodología

- **Análisis:** Estudio de la estructura del lenguaje definido para desarrollar el interprete.
- **Desarrollo:**
 - Implementación de analizador léxico con la herramienta jflex.
 - Construcción de la interfaz de usuario y lógica de simulación.
- **Visualización:** Exportar los reportes a tablas o con Graphviz para interpretación visual.
- **Validación:** Pruebas con cadenas válidas e inválidas.
- **Documentación:** manual técnico, manual de usuario y reportes.

11. Desarrollo de Habilidades Blandas

1. **Autogestión del tiempo:** Planificación y cumplimiento del cronograma
2. **Responsabilidad y compromiso:** Gestión completa del ciclo del proyecto
3. **Resolución de problemas:** Manejo de errores de parsing, ambigüedad gramatical, etc
4. **Reflexión personal:** Evaluación de resultados y mejoras.

12. Cronograma

Este cronograma describe las etapas clave del proyecto, con el proceso de asignación, elaboración y calificación de las tareas. Los estudiantes deberían seguir este plan para asegurar que el proyecto avance de manera organizada y cumpla con los plazos establecidos. Cada fase incluye la asignación de tareas, el tiempo estimado para su elaboración, y el momento de su finalización.

Tipo	Fecha Inicio	Fecha Fin
Asignación de Proyecto	04/02/2026	04/03/2026
Elaboración	04/02/2026	04/03/2026
Calificación	05/03/2026	07/03/2026

Nota: El proyecto deberá entregarse antes de las 23:59 de la fecha establecida, no se otorgaran prorrogas.

13. Rúbrica de Calificación

13.1 Detalle de la Calificación

*Hoja de calificación en proceso

13.2 Valores

En el desarrollo del proyecto, se espera que cada estudiante demuestre honestidad académica y profesionalismo. Por lo tanto, se establecen los siguientes principios:

1. **Originalidad del Trabajo**
 - Cada estudiante o equipo debe desarrollar su propio código y/o documentación, aplicando los conocimientos adquiridos en el curso.
2. **Prohibición de Copias y Plagio**
 - Si se detecta la copia total o parcial del código, documentación o cualquier otro entregable, la calificación será de **0 puntos**.
 - Esto incluye la reproducción de código entre compañeros, la reutilización de proyectos de semestres anteriores o el uso de código externo sin la debida referencia.
3. **Uso Responsable de Recursos Externos**
 - El uso de bibliotecas, frameworks y ejemplos de código externos está permitido, siempre y cuando se referencian correctamente y se comprendan plenamente.
4. **Revisión y Detección de Plagio**
 - Se podrán utilizar herramientas automatizadas y revisiones manuales para identificar similitudes en los proyectos.
 - En caso de sospecha, el estudiante deberá justificar su código y demostrar su desarrollo individual o en equipo. Si este extremo no es comprobable la calificación será de **0 puntos**.

Al detectarse estos aspectos se informará al catedrático del curso quien realizará las acciones que considere oportunas.

13.3 Comentarios Generales

Cada estudiante es responsable de llevar su computadora con el programa para optar a calificación.